

**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ**

**Отчёт по практике по дисциплине
«Математическое моделирование»**

**Планировщик операционной системы для
гетерогенной процессорной архитектуры**

Шаго Павел Евгеньевич

22.Б07-ПУ, 01.03.02 Прикладная математика и информатика

Проверил

доктор ф.-м.н., доцент

Михеев Сергей Евгеньевич

Санкт-Петербург

24 декабря 2025 г.

Содержание

Введение	3
1 Постановка	4
1.1 Физические предпосылки	4
1.2 Обозначения	5
1.3 Модель EEVDF	6
1.4 Математический аппарат	7
1.5 Модель A1349	9
Результаты	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11

Введение

Данная работа продолжает исследования [1], [2] и [3], связанные с планировщиком процессов в ядре Linux. В [3] был реализован алгоритм Earliest Eligible Virtual Deadline First (EEVDF) с использованием инфраструктуры sched_ext. Настоящая работа предлагает описание и реализацию алгоритма A1349 с помощью sched_ext.

Проведено сравнение с эталонной реализацией EEVDF в ядре Linux, а также разработанной в [3] реализацией EEVDF с использованием инфраструктуры sched_ext.

Несмотря на то, что алгоритм EEVDF считается высокопроизводительным и эффективным решением, работы [2] и [3] продемонстрировали, что обобщенный подход не всегда является хорошим решением.

Фреймворк sched_ext построен так, что позволяет практически без косвенных затрат загружать и выгружать новые алгоритмы во время работы операционной системы.

Современные процессорные архитектуры все чаще проектируются на основе гетерогенного принципа, при котором в рамках одного кристалла объединяются вычислительные ядра, различающиеся по уровню производительности. Такой подход позволяет более гибко распределять вычислительные нагрузки между ядрами, оптимизируя на аппаратном уровне использование ресурсов в зависимости от характера выполняемых задач.

Высокопроизводительные ядра ориентированы на обработку ресурсоемких и чувствительных к задержкам вычислений, тогда как энергоэффективные ядра предназначены для фоновых, периодических или менее требовательных операций.

Поиск и реализация алгоритмов планирования, направленных на более эффективное использование ресурсов процессора, представляет собой перспективное направление, так как более эффективные алгоритмы позволяют оптимизировать распределение вычислительного времени между задачами, что позволяет повысить производительность и отзывчивость операционной системы и уменьшить энергопотребление аппаратной системы.

1 Постановка

1.1 Физические предпосылки

Рассматривается аппаратная вычислительная система, включающая неоднородные вычислительные элементы, представленные ядрами различных типов (высокопроизводительными P-core и энергоэффективными E-core). Эти ядра отличаются по архитектуре, вычислительной мощности, энергопотреблению и эффективности выполнения задач, что обуславливает неоднородность доступных вычислительных ресурсов.



Рис. 1.1: Гетерогенный процессор Intel (целевая платформа)

Задача планирования заключается в максимально эффективном использовании ядер процессора с учётом их аппаратных особенностей и текущей нагрузки. Под эффективным использованием понимается такое распределение вычислительных процессов между ядрами, при котором достигается оптимизация заданного критерия, а также исключается нерациональное простаивание ресурсов. В условиях гетерогенной архитектуры данная задача усложняется необходимостью сопоставления требований процессов с возможностями конкретных ядер.

1.2 Обозначения

n – количество процессов

n_a – количество активных процессов

w_i – вес i -го процесса

q – квант времени

m – количество выделенных временных квантов

r – запрошенное процессом время на исполнение

r_{max} – максимально возможное время на исполнение

$r_i^{(k)}$ – продолжительность исполнения k -го запроса i -го процесса

$u_i^{(k)}$ – время которое фактически получает процесс i на k -ом запросе

t – момент в реальном времени

t_a – момент реального времени когда процесс i становится активным

t^- – время прямо перед наступлением события

t^+ – время сразу после наступления события

$A(t)$ – множество всех активных в момент t процессов

$W(t)$ – сумма весов всех активных процессов

$f_i(t)$ – доля процесса i во время t

$S_i(t_0, t_1)$ – время обслуживания которое должен получить процесс i в идеальной системе

$s_i(t_a, t)$ – время обслуживания которое клиент i на самом деле получает

$V(t)$ – виртуальное время системы

e – реальное подходящее время запроса

d – реальный дедлайн запроса

B – множество активных процессов дедлайн которых в $[e, d]$

C – множество активных процессов дедлайн которых больше d

$ve_i^{(k)}$ – виртуальное подходящее время k -го запроса i -го процесса

$vd_i^{(k)}$ – виртуальный дедлайн k -го запроса i -го процесса

$lag_i(t)$ – задержка обслуживания процесса i в момент времени t , $(S - s)$

d_{neg} – наибольший дедлайн среди всех процессов с отрицательными lag которые активны в момент времени t_1

1.3 Модель EEVDF

$$f_i(t) = \frac{w_i}{\sum_{j \in A(t)} w_j} \quad (\text{share})$$

$$V(t) = \int_{t_0}^t \frac{1}{\sum_{j \in A(\tau)} w_j} d\tau \quad (\text{virtual time})$$

$$S_i(t_1, t_2) = w_i(V(t_2) - V(t_1)) \quad (\text{ideal service})$$

$$lag_i(t) = S_i(t_0, t) - s_i(t_0, t) \quad (\text{service lag})$$

$$V(e) = V(t_0) + \frac{s_i(t_0, t)}{w_i} \quad (\text{virtual eligible time})$$

$$V(d) = V(e) + \frac{r}{w_i} \quad (\text{virtual deadline})$$

$$V(t^+) = V(t) + \frac{lag_j(t)}{\sum_{i \in A(t^+)} w_i} \quad (\text{join})$$

$$V(t^+) = V(t) - \frac{lag_j(t)}{\sum_{i \in A(t^+)} w_i} \quad (\text{leave})$$

$$V(t^+) = V(t) + \frac{lag_j(t)}{\sum_{i \in A(t^-)} w_i} - \frac{lag_j(t)}{\sum_{i \in A(t^+)} w_i} \quad (\text{weight change})$$

$$-r_{\max} < lag_i(t) < \max(r_{\max}, q) \quad (\text{lag bounds})$$

1.4 Математический аппарат

Модифирую алгоритм EEVDF путем добавления в него модели гетерогенного ядра. Смысл алгоритма EEVDF состоит в *взвешенном справедливом распределении ресурса*: реальное расписание должно как можно точнее имитировать идеализированную непрерывную систему, в которой каждый активный процесс получает долю ресурса пропорционально своему весу. При этом минимизируется абсолютное значение lag во времени.

Модель гетерогенного процессора

Пусть в момент времени t доступен набор вычислительных ядер $\mathcal{K}(t)$, а производительность ядра $c \in \mathcal{K}(t)$ равна $\rho_c > 0$ (нормированных единиц работы в секунду). Тогда суммарная мощность системы:

$$C(t) = \sum_{c \in \mathcal{K}(t)} \rho_c. \quad (1.1)$$

Пусть $A(t)$ — множество активных процессов, w_i — вес процесса i , а

$$W(t) = \sum_{j \in A(t)} w_j \quad (1.2)$$

— суммарный вес активных процессов.

Для описания решения планировщика введём индикатор исполнения

$$x_{i,c}(t) \in \{0, 1\}, \quad (1.3)$$

где $x_{i,c}(t) = 1$, если процесс i выполняется на ядре c в момент t .

Ограничения выполнимости (одно ядро — не более одного процесса, и процесс не параллелится на нескольких ядрах одновременно):

$$\sum_i x_{i,c}(t) \leq 1, \quad \forall c \in \mathcal{K}(t), \quad (1.4)$$

$$\sum_{c \in \mathcal{K}(t)} x_{i,c}(t) \leq 1, \quad \forall i \in A(t). \quad (1.5)$$

Фактически полученное обслуживание процесса i на интервале $[t_0, t_1]$ измеряется в *единицах работы*:

$$s_i(t_0, t_1) = \int_{t_0}^{t_1} \sum_{c \in \mathcal{K}(\tau)} \rho_c x_{i,c}(\tau) d\tau. \quad (1.6)$$

Идеализированная система

В идеальной системе каждый активный процесс получает долю мощности пропорционально своему весу. Доля процесса i :

$$f_i(t) = \frac{w_i}{W(t)}. \quad (1.7)$$

Идеальное (эталонное) обслуживание процесса i на интервале $[t_0, t_1]$:

$$S_i(t_0, t_1) = \int_{t_0}^{t_1} f_i(\tau) C(\tau) d\tau = \int_{t_0}^{t_1} \frac{w_i}{W(\tau)} C(\tau) d\tau. \quad (1.8)$$

Ошибка обслуживания (service lag)

Определим отставание (или опережение) процесса i относительно идеального распределения:

$$lag_i(t) = S_i(t_0, t) - s_i(t_0, t), \quad (1.9)$$

где t_0 — фиксированный момент начала отсчёта (например, начало наблюдения или момент входа процесса в систему).

Для сопоставимости процессов разных весов введём нормированную ошибку:

$$\delta_i(t) = \frac{lag_i(t)}{w_i}. \quad (1.10)$$

Целевая функция

Задача ставится как минимизация *наихудшего* по времени и по процессам нормированного отклонения от идеала:

$$\min_{\{x_{i,c}(t)\}} \max_{i \in A(t)} \left| \frac{lag_i(t)}{w_i} \right|. \quad (1.11)$$

1.5 Модель A1349

$$C(t) = \sum_{c \in \mathcal{K}(t)} \rho_c \quad (\text{total capacity})$$

$$f_i(t) = \frac{w_i}{W(t)}, \quad W(t) = \sum_{j \in A(t)} w_j \quad (\text{share})$$

$$S_i(t_0, t_1) = \int_{t_0}^{t_1} f_i(\tau) C(\tau) d\tau = \int_{t_0}^{t_1} \frac{w_i}{W(\tau)} C(\tau) d\tau \quad (\text{ideal service})$$

$$V(t) = \int_{t_0}^t \frac{C(\tau)}{W(\tau)} d\tau \quad (\text{virtual time})$$

$$S_i(t_1, t_2) = w_i (V(t_2) - V(t_1)) \quad (\text{ideal service via virtual time})$$

$$s_i(t_0, t_1) = \int_{t_0}^{t_1} \sum_{c \in \mathcal{K}(\tau)} \rho_c x_{i,c}(\tau) d\tau \quad (\text{actual service (work units)})$$

$$\text{lag}_i(t) = S_i(t_0, t) - s_i(t_0, t) \quad (\text{service lag (work units)})$$

$$Q_c = \rho_c q \quad (\text{work quantum on core } c)$$

$$Q_{\max} = q \cdot \max_{c \in \mathcal{K}(t)} \rho_c \quad (\text{max work quantum})$$

$$-p_{\max} < \text{lag}_i(t) < \max(p_{\max}, Q_{\max}) \quad (\text{lag bounds})$$

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in A(t^+)} w_i} \quad (\text{join})$$

$$V(t^+) = V(t) - \frac{\text{lag}_j(t)}{\sum_{i \in A(t^+)} w_i} \quad (\text{leave})$$

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in A(t^-)} w_i} - \frac{\text{lag}_j(t)}{\sum_{i \in A(t^+)} w_i} \quad (\text{weight change})$$

Результаты

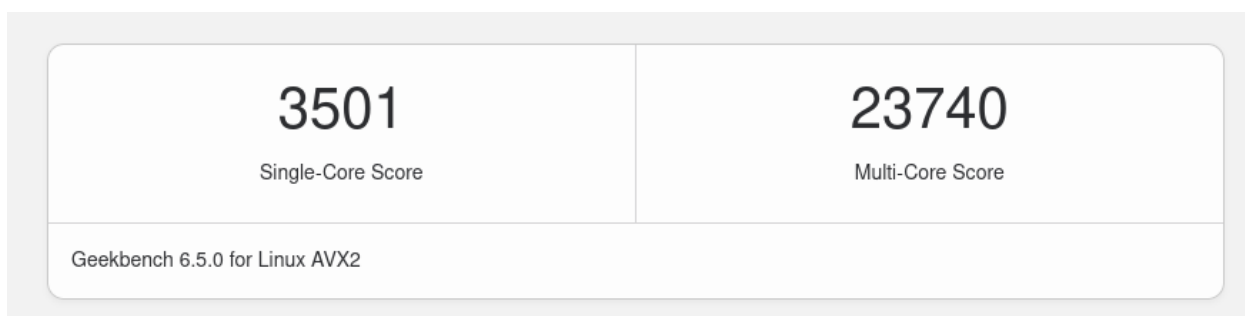


Рис. 1.2: EEVDF

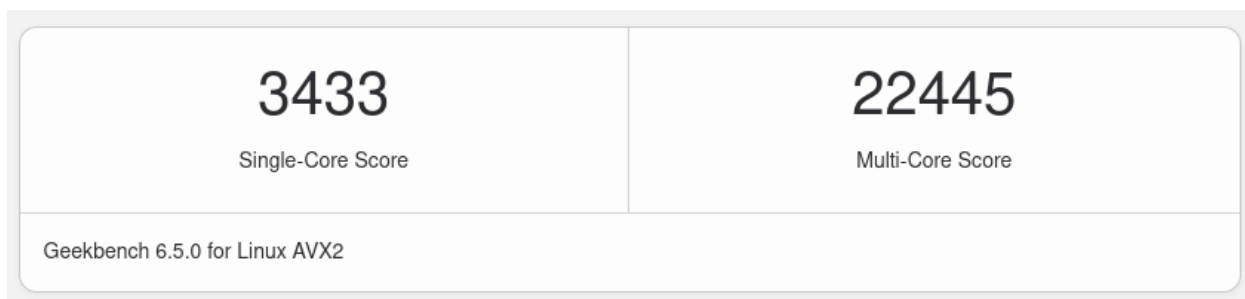


Рис. 1.3: EEVDF (sched_ext)



Рис. 1.4: A1349

В результате наблюдается снижение наивной производительности при использовании sched_ext по сравнению с эталонной реализацией EEVDF в ядре Linux.

При этом применение адаптированного к задаче планировщика A1349 (основанного на EEVDF) позволяет несколько поднять общую производительность.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Павел Е. Шаго Отчет о научно-исследовательской работе: Планировщики 6. и. 2024
- [2] Павел Е. Шаго Отчет о научно-исследовательской работе: Планировщики 2 б. и. 2025
- [3] Павел Е. Шаго Отчет о научно-исследовательской работе: Планировщики 3 б. и. 2025
- [4] Joseph Y-T. Leung Handbook of Scheduling: Algorithms, Models, and Performance Analysis CRC Press 2004
- [5] Michelle Galin, Anna Hedblom Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [6] Jinsong Mao, Benjamin E. Ujcich, Shiqing Ma Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [7] Xinbo Wang, Shian Jia, Ziyang Huang, Jing Cao, Mingli Song Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [8] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [9] A1349 Pavel Shago [Электронный ресурс]. – Режим доступа: <https://github.com/shgpavel/A1349> – Дата обращения 16.12.2025.
- [10] Andrew S. Tanenbaum, Herbert Bos MODERN OPERATING SYSTEMS Pearson Education 2023
- [11] Liz Rice Learning eBPF O'Reilly Media, Inc 2023
- [12] Torvalds L. Linux Kernel [Электронный ресурс] / L. Torvalds. – Режим доступа: <https://github.com/torvalds/linux> – Дата обращения 04.12.2025.
- [13] sched-ext/scx [Электронный ресурс]. – Режим доступа: <https://github.com/sched-ext/scx> – Дата обращения 07.12.2025.
- [14] lkml [Электронный ресурс]. – Режим доступа: <https://lkml.org> – Дата обращения 24.11.2025.